DEPARTMENT OF MECHATRONICS & CONTROL ENGINEERING

UNIVERSITY OF ENGINEERING & TECHNOLOGY

Mobile ROBOTICS LAB TWO

SUBMITTED BY:

Sahra Irshad 2022-MC-32

SUBMITTED TO:

Mam Maria

Lab Report: Familiarization with rqt and ROS 2 Services

Objective

The objective of this lab was to gain practical understanding of the ROS 2 graphical tool **rqt** and to learn how ROS 2 nodes communicate using **topics and services**. The experiment also aimed to explore the **Turtlesim simulation environment** and observe how different nodes interact within the ROS 2 ecosystem. By using rqt plugins such as **Node Graph** and **Service Caller**, students were able to visualize active nodes, analyze topic communication, and execute service calls.

Introduction

Robot Operating System (ROS 2) provides a flexible framework for developing robotic applications. In ROS 2, communication between different components of a robotic system occurs through **nodes**, **topics**, and **services**.

Nodes are individual programs performing specific tasks.

Topics enable continuous communication between nodes using a publish–subscribe mechanism.

Services allow nodes to request and receive responses for specific tasks.

The **rqt tool** is a graphical user interface provided by ROS 2 that allows users to monitor and control the system visually. In this lab, the **Turtlesim simulator** was used to demonstrate these concepts by controlling virtual turtles.

Procedure

1. Launching the Turtlesim Node

The first step was to launch the turtlesim simulation environment using the ROS 2 command line. This opened a graphical window displaying a turtle that can be controlled through ROS topics.

2. Opening the rqt Tool

The command rqt was executed to start the ROS graphical interface. This tool provides several plugins that allow users to visualize and interact with nodes, topics, and services.

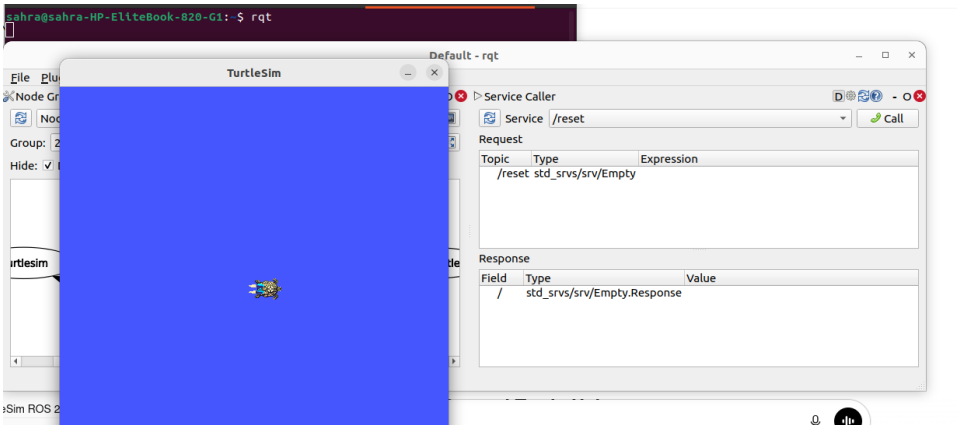
3. Viewing Node Graph

The **Node Graph plugin** in rqt was used to visualize the ROS 2 communication structure. This plugin displayed the active nodes and the topics connecting them. The turtlesim node and its associated topics such as velocity and pose were observed.

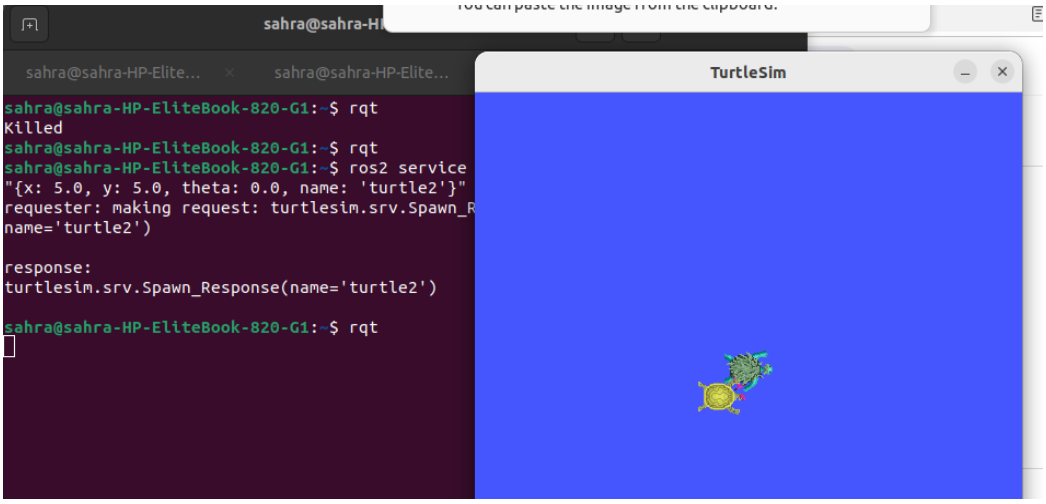
4. Using ROS 2 Services

The **Service Caller plugin** was used to interact with services provided by turtlesim.

First, the /reset service was called. This service resets the turtle to its initial position in the simulation environment.

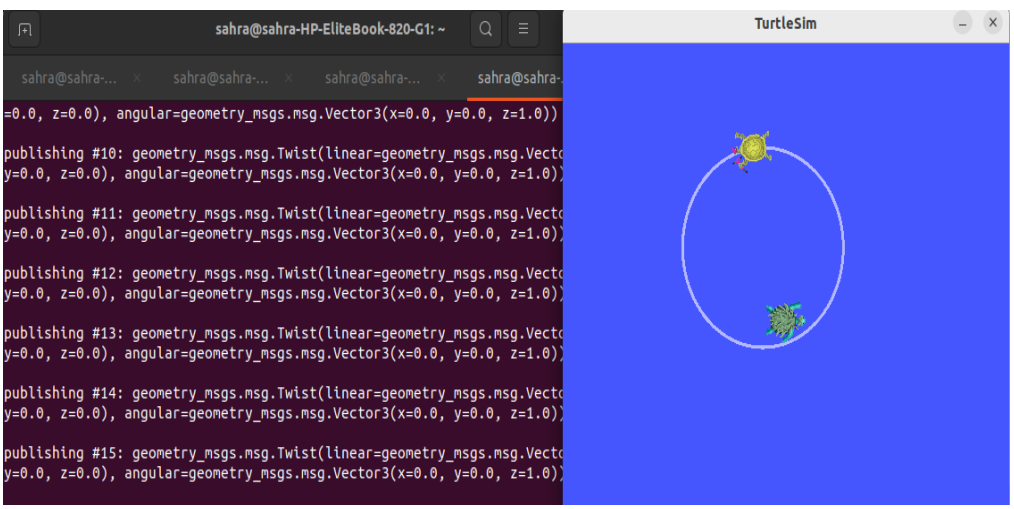


Next, the /spawn service was used to create a second turtle. Parameters such as **x position, y position, orientation (theta), and name** were specified to generate the new turtle in the simulation window.



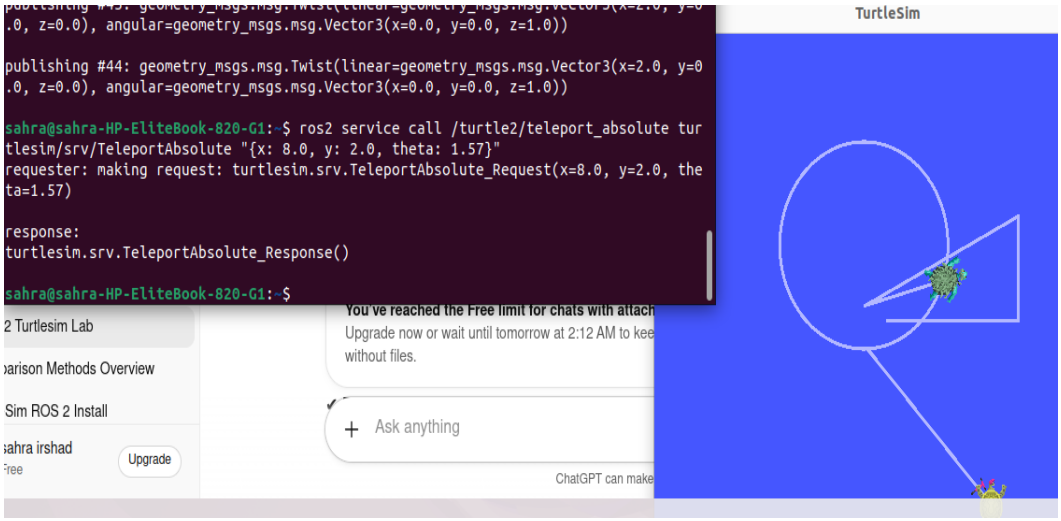
5. Controlling Multiple Turtles

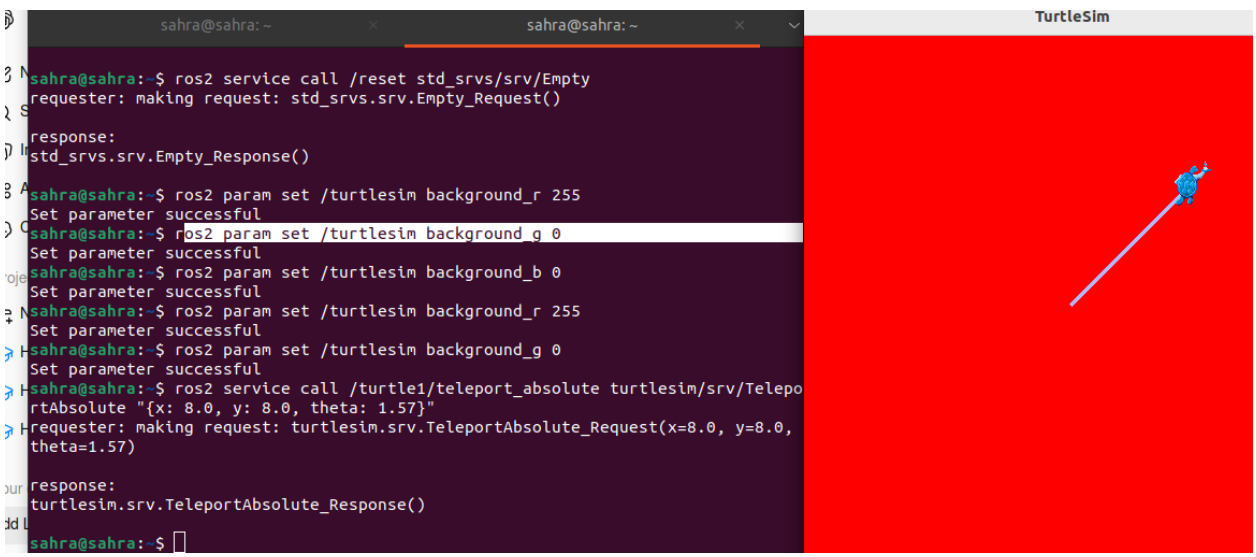
After spawning the second turtle, the topic /turtle2/cmd_vel was identified. This topic allows velocity commands to be sent to the second turtle. By publishing velocity commands to this topic, the second turtle could be moved independently from the first turtle.



6. Testing Additional Services

Additional services provided by turtlesim were explored. These included services for **teleporting the turtle to different positions** and **changing the background color of the simulation window**.





TELEPORTING AND BACKGROUND CHANGE

Observations

- *Topics continuously publish data such as velocity and pose.*
- *Services perform one-time actions such as reset and spawn.*

- *rqt provides a graphical way to monitor and interact with the ROS 2 system.*
- *Multiple turtles can be controlled independently using separate topics.*

<u>Conclusion</u>

This lab helped in understanding the practical implementation of nodes, topics, and services in ROS 2. The rqt tool made it easier to visualize and interact with the system in real time.